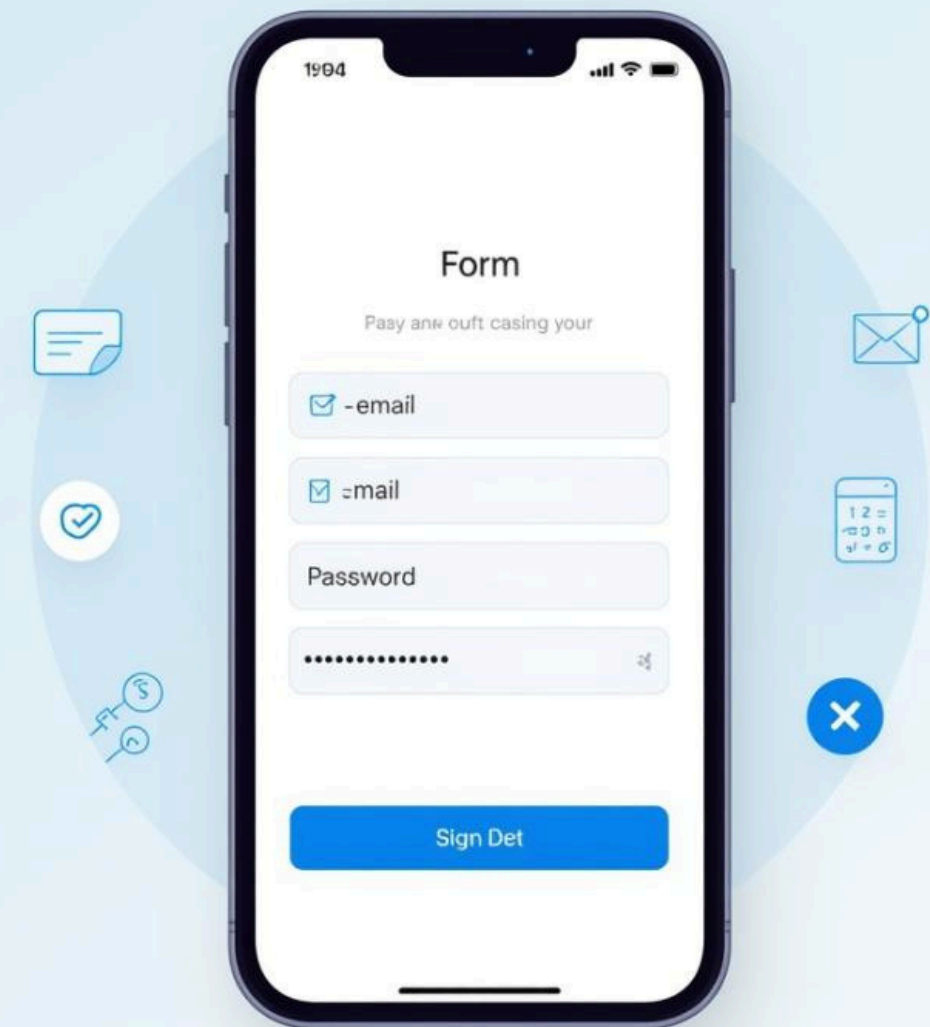
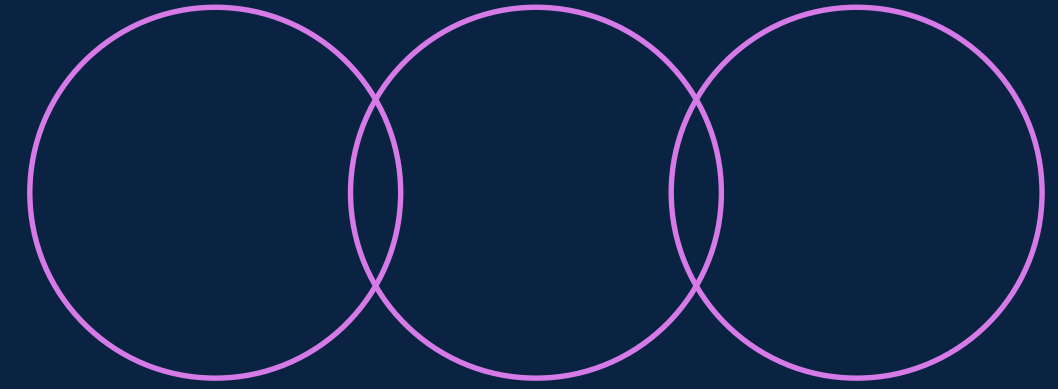


Gestion des formulaires en mobile

Présenté par Meriem el mecaniqui & Samira kibous



Introduction



Un formulaire mobile permet à l'utilisateur de saisir et envoyer des informations, comme un email, un mot de passe ou un numéro de téléphone.

Les formulaires sont essentiels dans les applications mobiles car ils permettent :

- la connexion et l'inscription des utilisateurs,
- la collecte d'informations importantes,
- la réalisation d'actions clés, comme les paiements ou la mise à jour de profil.

Dans cette présentation, nous verrons comment gérer les champs (inputs), valider les formulaires, utiliser des bibliothèques de validation, afficher les erreurs et suivre les bonnes pratiques pour offrir une expérience utilisateur optimale.

Spécificités mobiles

Expérience utilisateur

L'expérience utilisateur pour les formulaires mobiles doit être intuitive, fluide et adaptée aux interactions rapides sur des écrans réduits.

Ergonomie mobile

La disposition et la taille des champs doivent être optimisées pour une utilisation facile, même sur des écrans de petite taille.

Gestion du clavier

Une gestion efficace du clavier mobile est essentielle pour garantir une saisie facile et sans interruptions pour l'utilisateur.

Disposition générale

Une disposition claire et logique des éléments du formulaire aide à la compréhension et encourage les utilisateurs à compléter le processus.

Gestion des champs

Utilisation de TextInput

TextInput dans React Native permet de saisir des données (email, mot de passe, etc.) et utilise des props essentielles comme `value`, `onChangeText`, `placeholder` et `secureTextEntry`.

Optimisation

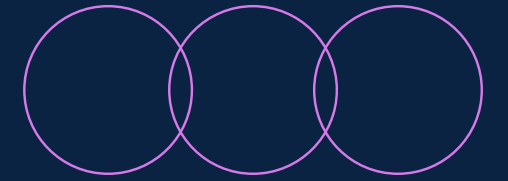
viter les re-renders inutiles, utiliser `React.memo` et choisir entre des inputs contrôlés (controlled) ou non contrôlés (uncontrolled).

Gestion de l'état

`useState` permet de stocker les valeurs des champs, tandis que `useReducer` est utile pour gérer des formulaires plus complexes

Gestion du focus

On utilise `ref` dans React Native pour accéder directement à un champ `TextInput`, ce qui permet de gérer le focus, passer automatiquement au champ suivant après la saisie, ou déclencher des actions spécifiques sur le champ via `ref.current.focus()`.



Validation des formulaires

Validation manuelle

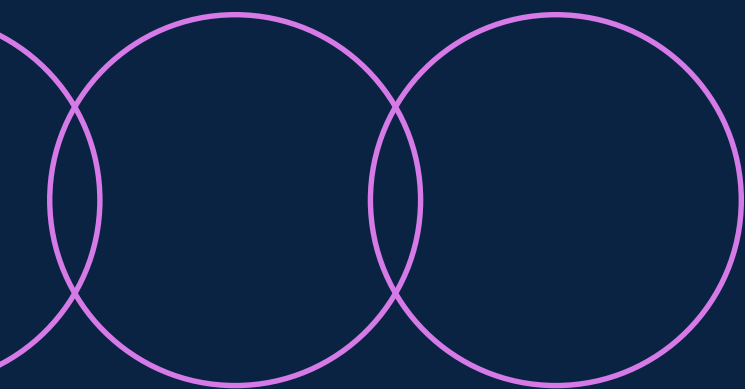
La **validation manuelle** permet un contrôle personnalisé des champs d'entrée, garantissant que les utilisateurs fournissent des informations correctes. En définissant des règles claires, vous pouvez identifier les erreurs spécifiques et fournir des retours immédiats, améliorant ainsi l'expérience utilisateur.

Vérification obligatoire

La **vérification des champs obligatoires** assure que les utilisateurs remplissent toutes les informations essentielles avant de soumettre le formulaire. Cela aide à éviter les soumissions incomplètes et renforce la qualité des données collectées, augmentant l'efficacité de l'application.

Librairies pour formulaires

Outils essentiels pour la gestion des formulaires mobiles



Formik

Formik simplifie la gestion des formulaires en React Native, offrant un cadre clair pour les champs et la validation des données utilisateur.

React Hook Form

React Hook Form se concentre sur la performance, utilisant des hooks pour une gestion dynamique des formulaires et une validation efficace.

Yup

Yup est une librairie de validation qui permet de définir des schémas déclaratifs, facilitant ainsi la validation des données dans les formulaires.

Gestion des erreurs

Messages d'erreur

Des messages d'erreur **clairs et visibles** permettent aux utilisateurs de comprendre rapidement les problèmes rencontrés lors de la saisie des données.

Messages de succès

Informar les utilisateurs avec des **messages de succès** après validation rassure et encourage l'interaction continue avec le formulaire.

Indication visuelle

Utiliser des **couleurs et icônes** pour indiquer les champs invalides aide à attirer l'attention des utilisateurs sur les erreurs spécifiques à corriger.

Importance du feedback

Le feedback efficace est crucial pour une **expérience utilisateur** positive, aidant à maintenir l'engagement et à réduire la frustration.

Bonnes pratiques

KeyboardAvoidingView

Utiliser KeyboardAvoidingView pour éviter que le clavier ne masque les champs de saisie, améliorant ainsi l'accessibilité et l'expérience utilisateur.

keyboardType

Utiliser keyboardType approprié pour chaque champ de saisie, facilitant l'entrée des données et assurant une meilleure expérience de saisie utilisateur.

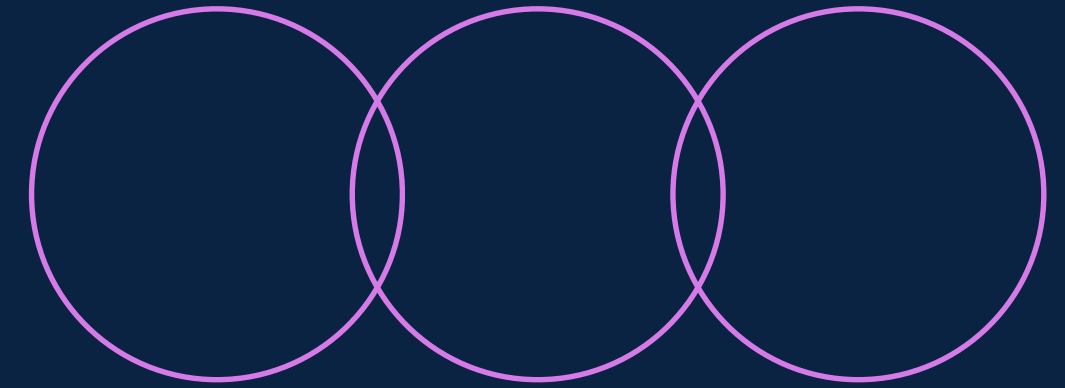
ScrollView

Intégrer ScrollView pour permettre le défilement des formulaires longs, garantissant que tous les champs sont accessibles et visibles à l'utilisateur.

Accessibilité

Assurer l'accessibilité en ajoutant des labels, un focus approprié et une navigation par clavier, permettant à tous les utilisateurs d'interagir facilement avec le formulaire.

Conclusion



En conclusion, la gestion des formulaires est un élément essentiel des applications mobiles, car elle permet de collecter et traiter les informations des utilisateurs de manière efficace et sécurisée.

Pour créer des formulaires performants, il est important de :

- Gérer correctement les champs (inputs) et leur état.
- Valider les données avant l'envoi.
- Afficher clairement les erreurs pour guider l'utilisateur.
- Suivre les bonnes pratiques, comme l'utilisation de `KeyboardAvoidingView`, de `ScrollView` et de bibliothèques de validation comme `Formik` ou `React Hook Form`.

En respectant ces principes, on améliore l'expérience utilisateur et la fiabilité de l'application.